

EECE4632 Hardware-Software Codesign for FPGA Based Systems

Preliminary Project Report

David Noble & Clara Small

April 14, 2026

Abstract

For our Final Project, we are going to design a Guitar Pedal with audio processing algorithms on the AUP-ZU3 board. We will use the Audio Codec on the board to input the sound, and design specified algorithms to control the sound in C. Then we will create overlays to run the algorithms on the hardware, and compare it to the speed of the algorithms in Python software. We will output the modified sound through the board's DAC to a speaker, and hear the changes in real time.

What we implemented for Project Update 1

We completed the algorithms for hard and soft clipping distortion, tremolo, echo, and chorus in Python for our software implementation. We created a Vitis .cpp file to test our wav file input and output on the AUP-ZU3 audio codec.

What we implemented for Project Update 2

We focused a LOT on testing the input wav file and output audio codec on the board, creating an overlay from the Vitis code and altering the Vivado design many times. We changed our design to utilize AXI Stream instead of AXI Lite and created custom outputs for the board in the Vivado design.

After we still could not get this to work, David kept focusing on debugging the Vivado, finding issues with the I2C drivers and I2S connections. Clara started writing the C++ code for the effects, translating them from our existing Python code. She got the hard and soft clipping distortion working as well as the echo, verifying them with a testbench with a simulated sine wave.

Since Project Update 2

We finished implementing the tremolo and chorus effects in C++ and decided to return the output audio to the Jupyter notebook instead of using the audio codec, as it was still not working. To do this we just bypassed all the blocks used for the codec, and had it wired straight back to the output. We also added a new GPIO ip block that we could use to access the on board switches, that we wanted to integrate with our vitis code. We also worked on some hardware exploration to optimize our design, which is explained below.

Testing and Results

The original recording and recordings after implementing the guitar effects can be found here: https://drive.google.com/drive/folders/1Ga9601ZqkzmuVqzdtSRalNAjPa02avUf?usp=drive_link

Software times:

Time taken for Hard Clip: 0.7541706999763846 seconds

Time taken for Soft Clipped: 0.6168003000202589 seconds

Time taken for Delay: 0.3625282000284642 seconds

Time taken for Sin Tremolo: 0.6020116999861784 seconds

Time taken for Chorus: 1.475382599979639 seconds

Hardware times:

Hard clipping 0.0383 s - 19.6 x Faster

Soft clipping 0.0386 s - 16 x Faster

Echo 0.0381 s - 9.5 x Faster

Chorus 0.0392 s - 37.6 x Faster

Tremolo 0.0381 s - 15.8 x Faster

Hard + echo 0.0379 s - 10.2 x Faster

Soft + chorus 0.0379 s - 38 x Faster

Echo + chorus 0.0379 s - 39.3 x Faster

All of them 0.0380 s - 100 x Faster

All - tremolo 0.0383 s - 83.7 x Faster

Comparing our results from software to hardware, we see significant improvements in speed, an average improvement of around 20x over the software implementation. If this were to be used as an actual guitar pedal, the delays we see in hardware are small enough that the user would likely not notice any latency, while they almost certainly would if the software implementation were used.

One interesting result is that regardless of the effect, the execution time remains nearly constant for the hardware implementation. This is likely because of how our pipeline was designed and optimized. Every sample passes through each stage whether the effect is active or not, so the pipeline depth is the same as if all effects were enabled. We did not implement a shortcut for the data to bypass unused stages. Doing so might decrease the time for individual effects, but would likely increase the hardware resource cost.

Hardware Exploration

Our code originally used the `cmath` library with many floating point variables, as we were focused on getting code we could start from. This had a huge area, so we updated our code to use `hls_math` instead of `cmath` and `fixed<32,16>` variables instead of floating points. This massively decreased our area. There were few variables we had to cast as floating points, because the math functions like `tanh` and `sinf` required this.

All of our code used loop flatten off and HLS pipeline `II=1` because our effects process one audio sample per clock cycle in a streaming fashion, and pipeline `II=1` ensures this throughput is maintained. Loop flattening was disabled because our loop structures use AXI Stream reads with `TLAST` termination, and flattening could introduce stalls or interfere with the stream interface behavior.

One thing of note is the pragmas used for our buffer arrays: `bind storage and dependence`. These were added to resolve iteration interval warnings we were getting during HLS synthesis. `Bind storage` defines how many read and write ports are available per cycle. For the echo buffer, this is set to one read and one write, as it only needs to write the current sample to a new position and read from an older one. The chorus buffer requires two read ports because of the interpolation between two delay taps. The `dependence false` pragma tells Vitis that reads and writes on each buffer are always well separated in address space, which it would not have been able to determine on its own. Without this hint, HLS would conservatively assume a dependency and stall the pipeline, breaking our `II=1` target.

Then, we started playing around with improving our math. We decreased the number of divisions in our code by making a division constant before calling it within a loop and using `if` statements for the modulus operator. We tried using a cubic function for soft clipping distortion instead of `tanh`, but the results were exactly the same for both so we went back to using `tanh`. The last thing we tried was using the `fixed<32,16>.toint()` instead of just casting these variables to ints. This one had a slightly bigger area but we ultimately decided on this implementation to get rid of the quantization in the tremolo effect.

All of our designs had around the same speed, and can be seen below.

1. Using `cmath` and floating points, loop flatten off, pipeline `II=1`
Area: 110,819 blocks
Speed: 7.445 ns
2. Using `hls_math` and `fixed<32,16>`, loop flatten off, pipeline `II=1`
Area: 68974 blocks
Speed: 7.152 ns
3. Using `hls_math` and `fixed<32,16>`, loop flatten off, pipeline `II=1`, less division in soft clipping
Area: 48694 blocks
Speed: 7.057 ns

4. Using `hls_math` and `fixed<32,16>`, loop flatten off, pipeline II=1, less division in soft clipping, using cubic for soft clipped instead (EXACTLY THE SAME AS ABOVE)
Area: 48694 blocks
Speed: 7.057 ns
5. Using `hls_math` and `fixed<32,16>`, loop flatten off, pipeline II=1, less division in soft clipping, `to_int()` (SAME AS 3 BUT WITH `FIXED_T` FIX)
Area: 65166 blocks
Speed: 7.236 ns

Lessons Learned/Future Considerations

Throughout this project, we learned that creating multiple bitstreams for trial and error can take upwards of 30 minutes. Other issues we ran into when experimenting include the AXI pipelining freezing when the audio codec didn't work. This made it difficult to test both the audio codec connection and the guitar pedal effects, and we didn't have enough time to do them separately. Next time, we would definitely do these separately, or create a better method of testing the pedal effects in Vitis so we didn't have to rely on listening to the output sound.

We also learned a lot about how to code for hardware, improving our skills with hardware exploration and the best designs for our board. The fixed point variables really helped our design in terms of area, and so did the `hls_math` library (which we previously didn't know existed). We also learned how to connect the board switches to our design in Vivado and Vitis to use them as inputs.

The next steps in our project would be to get the audio codec working, and to fix the tremolo quantization.

How we used AI

We used AI to write a testbench to verify soft clipping, hard clipping, and echo in Vitis. This used a simulated sine wave and was right before the project 2 deadline. We wanted a quick check before testing these effects with the actual wav file in hardware.

We also used AI to give some suggestions for hardware exploration. For example, it gave us some ideas for improving our math functions, like eliminating divisions and the modulus operator. It also gave the suggestion of using the cubic function instead of the tanh function.

We used AI to help us debug, mostly with the Vivado project. It was used to help try and read Vivado error messages, and give ideas on how we could set up our codec. It also helped to tell us which IP to use to access the switches. In the end, it would likely have been more beneficial to mimic other projects directly than to have AI try to coach us on how to set up the codec.

Further Reading

- Reddit thread about audio connections in Vivado:
https://www.reddit.com/r/FPGA/comments/1i15pr9/help_with_i2s_ip_core_setup_in_vivado_no_processor/?solution=1a57d52f0415927a1a57d52f0415927a&js_challenge=1&token=bbbe4bflc9a2b5160829c4be34da5861f4a0301af8b00f59661d9425b91284b5